

Encoder-Decoder-sequence-to-sequence Architecture

- Sequence to Sequence (often abbreviated to seq2seq) models is a **special class** of Recurrent Neural Network architectures.
- We typically use **seq2seq** models to solve complex Language problems like
 - Machine Translation
 - Question Answering
 - creating Chatbots
 - Text Summarization

Machine Language Translation

*Les modèles de séquence
sont super puissants*

Sequence Model

*Sequence models are super
powerful*

English ▾



Chinese (Traditional) ▾

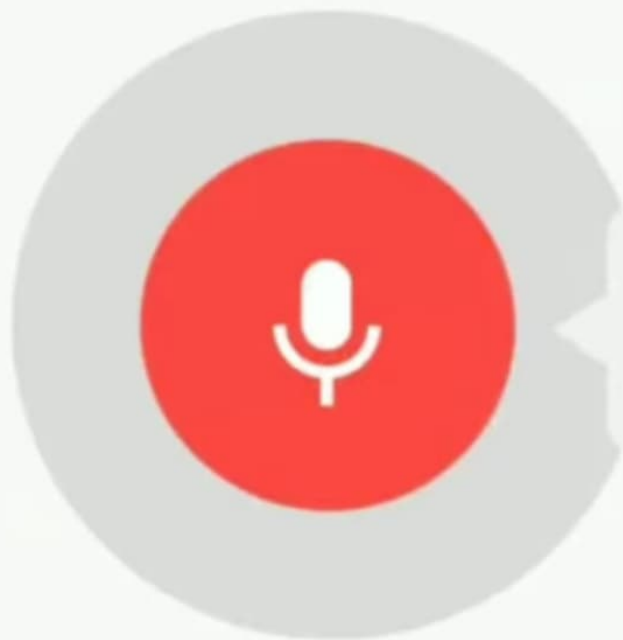


How are you? Edit

你好嗎？

Nǐ hǎo ma?

Speech recognition



"Ok Google"

Video captioning

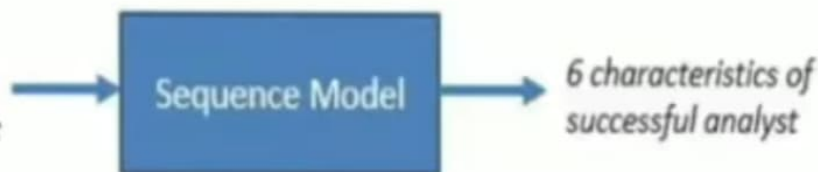


A herd of zebras are walking in a field.

Text Summarization

A strong analyst have 6 main characteristics. One should master all 6 to be successful in the industry :

1.
2.



6 characteristics of successful analyst

Chatbot

How are you doing today?



I am doing well. Thank you.
How are you doing today?

- **Seq2Seq** model can be used as a **solution** to any sequence-based problem, especially ones where the **inputs** and **outputs** have **different sizes** and **categories**.
- A sequence to sequence model aims to **map** a fixed-length input with a fixed-length output where the length of the input and output may differ.

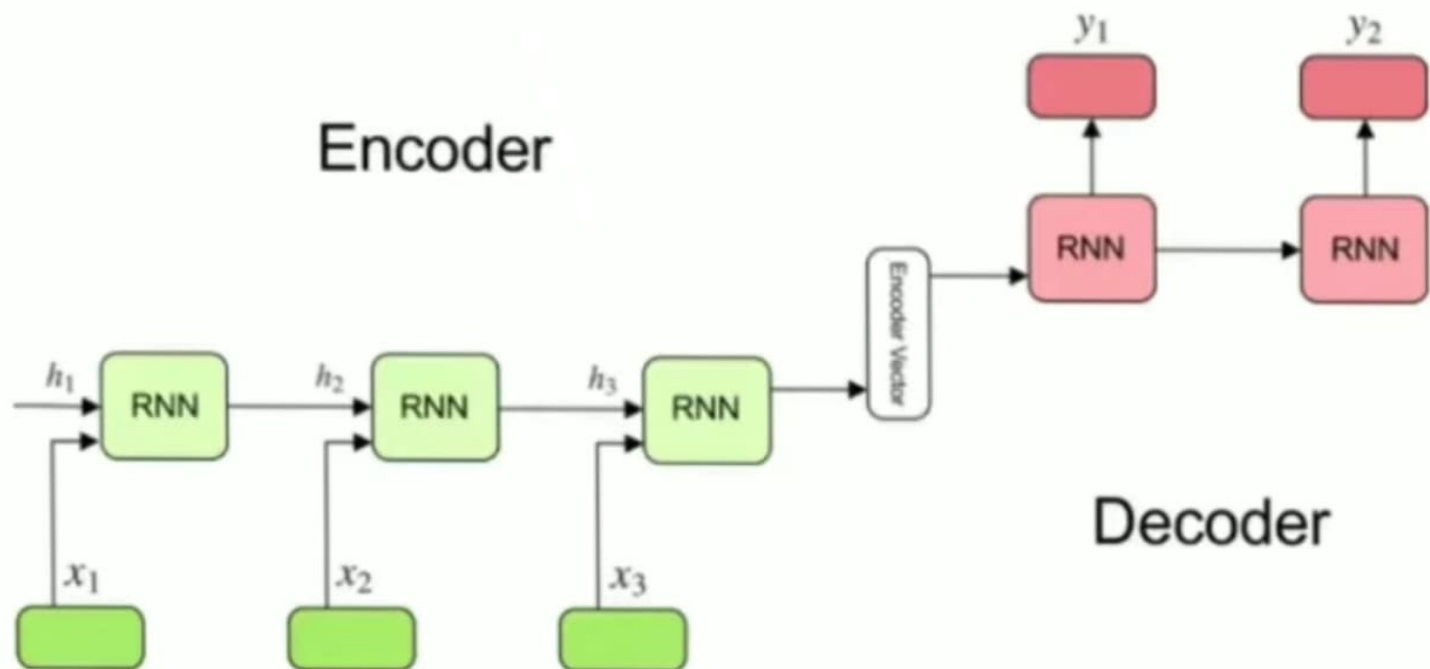
For example, translating

“**What are you doing today?**” from English to Chinese

has input of **5 words** and **output of 7 symbols** (今天你在做什麼?).

- The sequence to sequence tasks are very challenging as the **size of the inputs and outputs vary**.
- To tackle *many-to-many* sequence prediction problems, researchers have explored and find a way in the form of **Encoder-Decoder** Architecture.

How the Sequence to Sequence Model works?



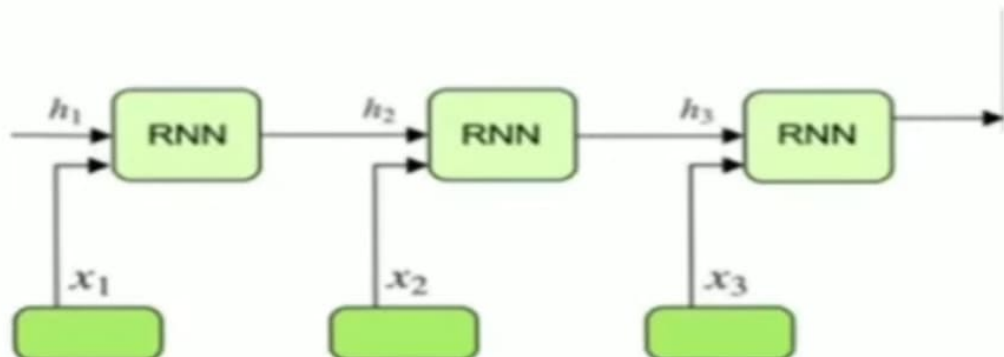
Encoder-decoder sequence to sequence model

The model consists of 3 parts:

1. **Encoder,**
2. **Intermediate vector or encoder vector or Hidden Vector**
3. **Decoder.**

- The **Encoder** will **convert** the **input sequence** into a **single dimensional vector** (hidden vector).
- The **decoder** will **convert** the **hidden vector** into the **output sequence**.

Encoder



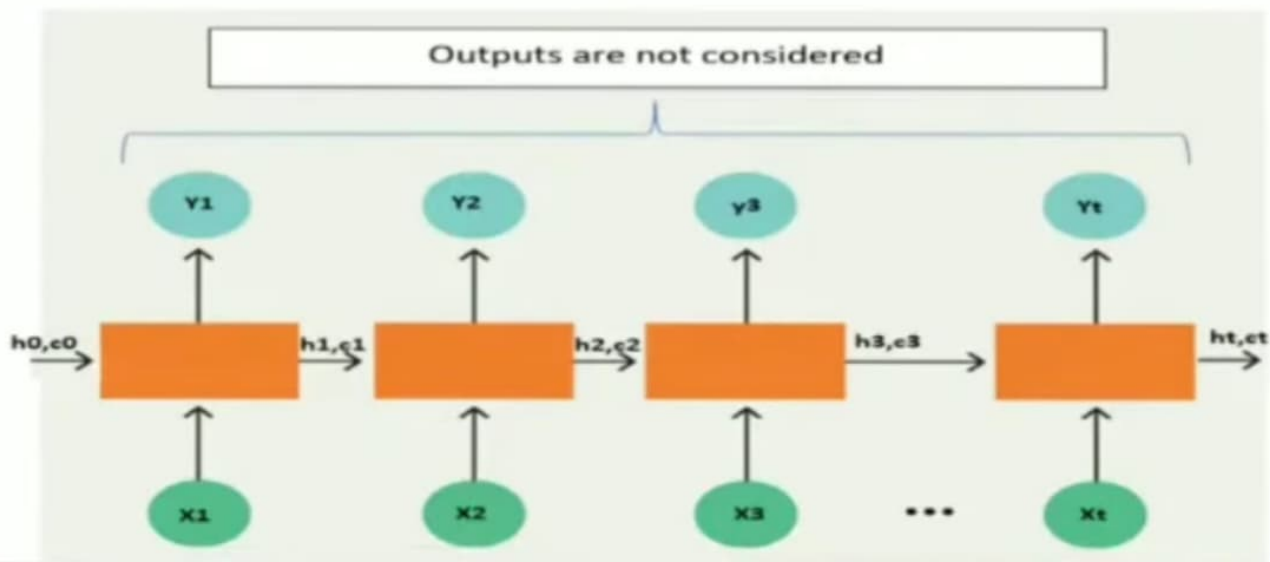
- Multiple RNN cells (LSTM or GRU cells for better performance) can be stacked together to form the encoder. RNN reads each inputs **sequentially**.
- For **every timestep** (each input) t , the **hidden state** (hidden vector) h is **updated** according to the **input** at that **timestep** $X[i]$.

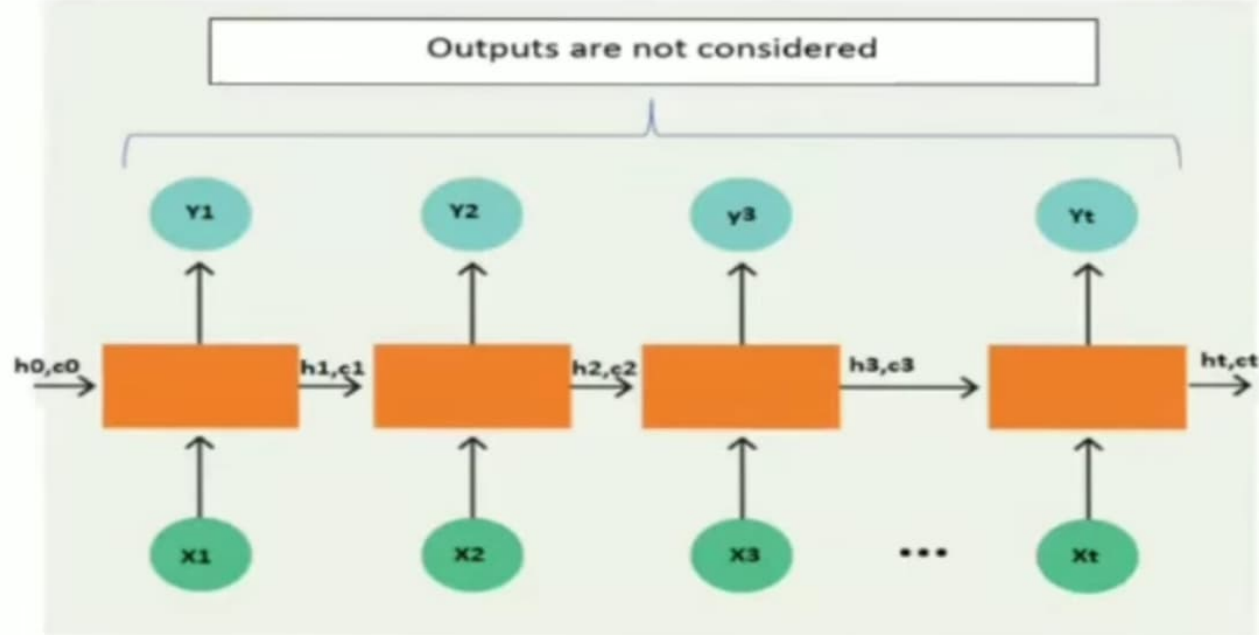
- After **all** the **inputs** are **read** by **encoder** model, the **final hidden state** of the model represents the **context/summary** of the whole input sequence.

Example:

- Consider the input sequence "**I am a Student**" to be encoded.
- There will be totally 4 timesteps (4 tokens) for the Encoder model.
- At **each time step**, the **hidden state h** will be **updated** using the **previous hidden state** and the **current input**.

- At the **first** timestep t_1 , the previous hidden state h_0 will be considered as **zero** or **randomly** chosen.
- So the first RNN cell will update the current hidden state with the **first input** $[x_1]$ and h_0 .





Each layer outputs **two** things —

1. Updated hidden state and
2. Output for each stage.

The **outputs** at each stage are **rejected** and **only the hidden states** will be **propagated** to the **next layer**.

The hidden states h_i are computed using the formula:

$$h_t = f(W^{(hh)} h_{t-1} + W^{(hx)} x_t)$$

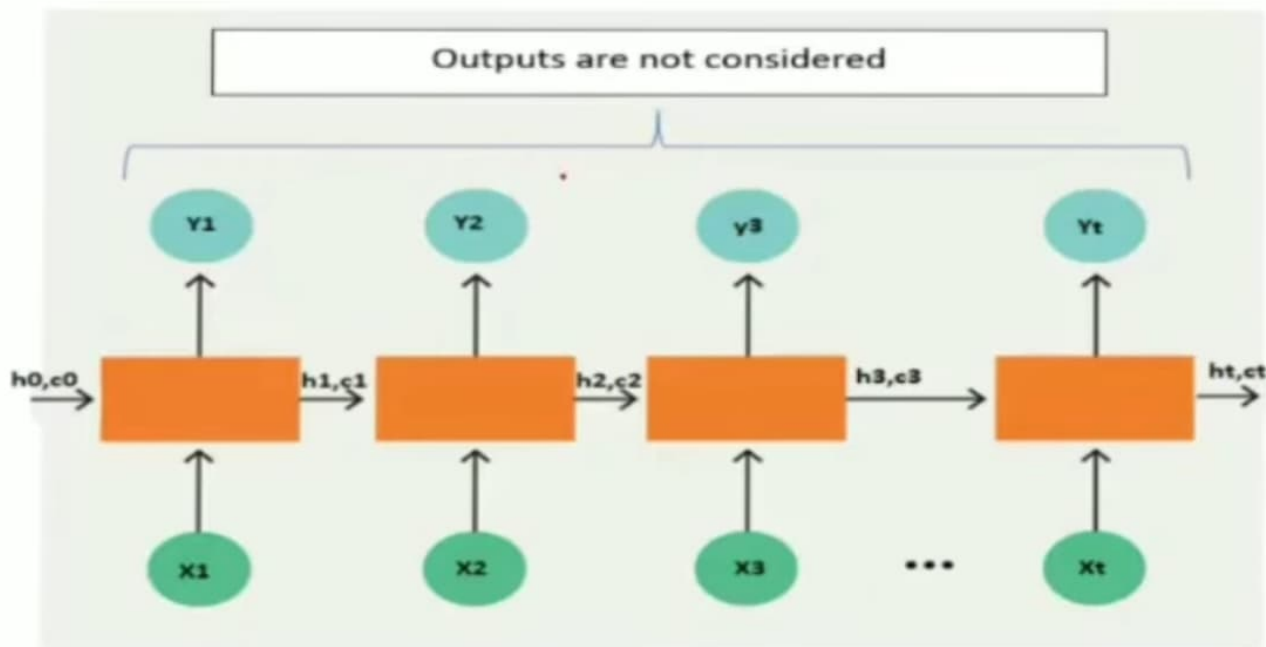
W_{hh} - weight at the recurrent neuron

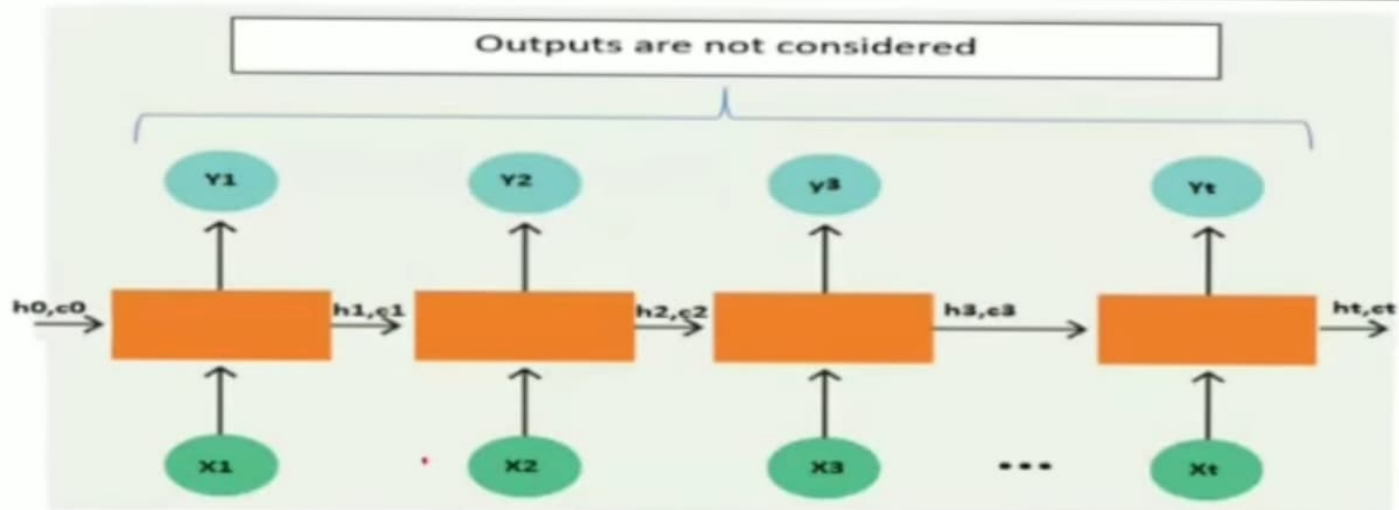
h_{t-1} - Previous State

W_{hx} - input neuron weight

x_t - current input

- At second timestep t_2 , the **hidden state h_1** and the **second input $X[2]$** will be given as input, and the hidden state h_2 will be updated according to both inputs.
- This happens for all the four stages wrt example taken.



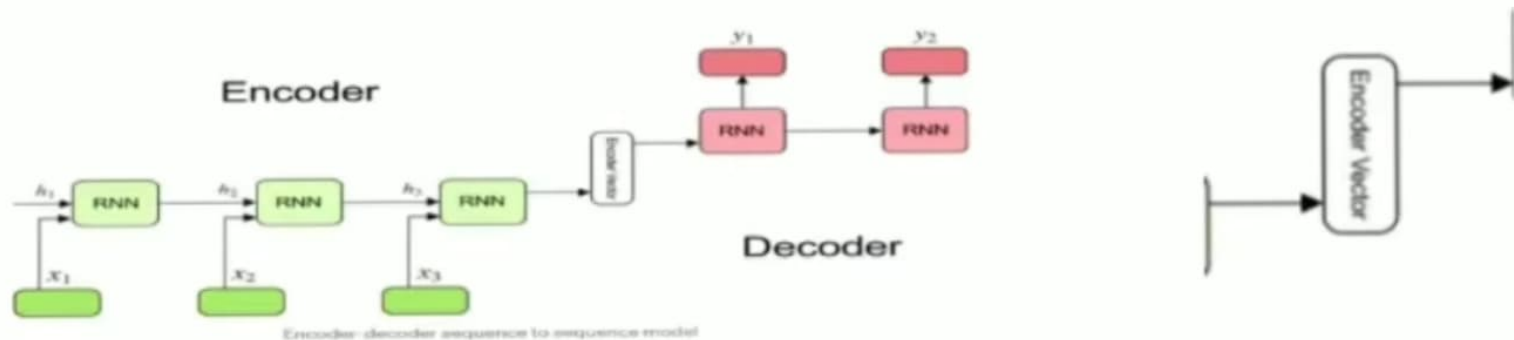


The LSTM reads the data, **one sequence after the other**. Thus if the input is a sequence of length ' t ', then LSTM reads it in ' t ' time steps.

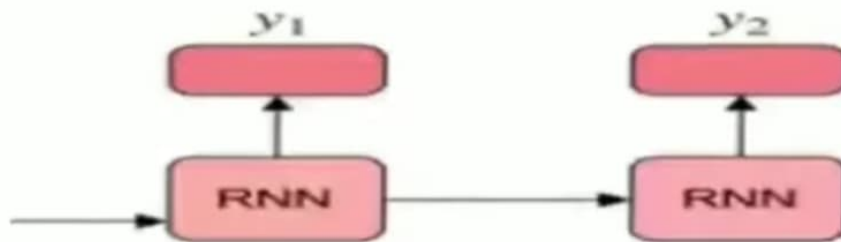
1. x_i = Input sequence at time step i .
2. h_i and c_i = LSTM maintains **two** states (' h ' for **hidden** state and ' c ' for **cell** state) at each time step. **Combined together** these are internal state of the LSTM at **time step i** .
3. y_i = **Output sequence** at time step i .

Encoder Vector

- This is the **final hidden state** produced **from** the **encoder** part of the model.
- It is calculated using the formula .
$$h_t = f(W^{(hh)} h_{t-1} + W^{(hx)} x_t)$$
- This **vector** aims to **encapsulate** the information **for all input elements** in order to **help** the **decoder** make **accurate predictions**.
- It **acts** as the **initial hidden state** of the **decoder** part of the model.



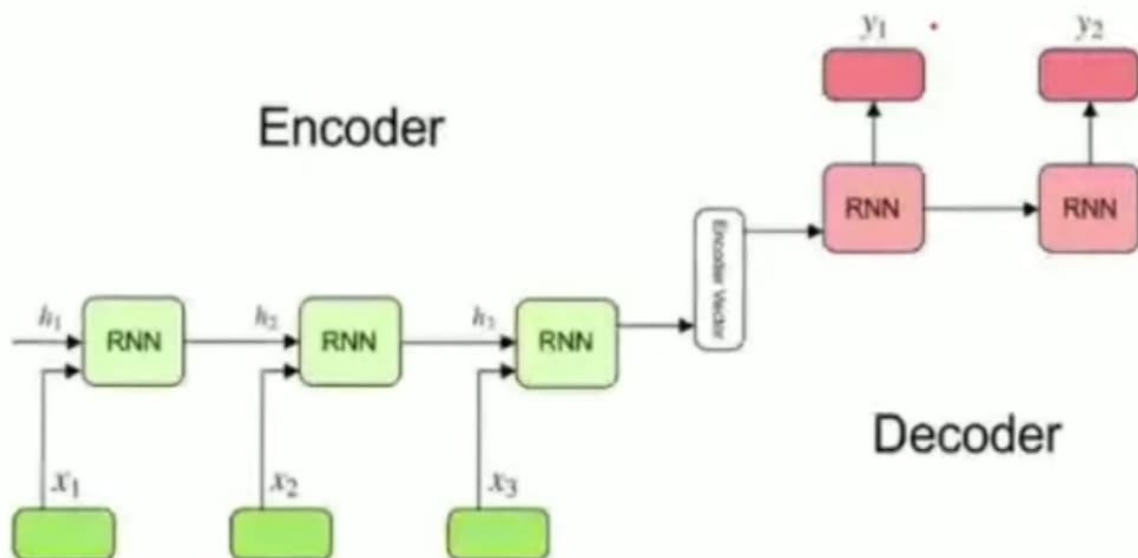
Decoder



Decoder

.

- A **stack** of several recurrent units where **each unit predicts an output y_t** at a time step **t** .
- The **input** for the **decoder** is the **final hidden vector** obtained at the **end of encoder** model.



Encoder-decoder sequence to sequence model

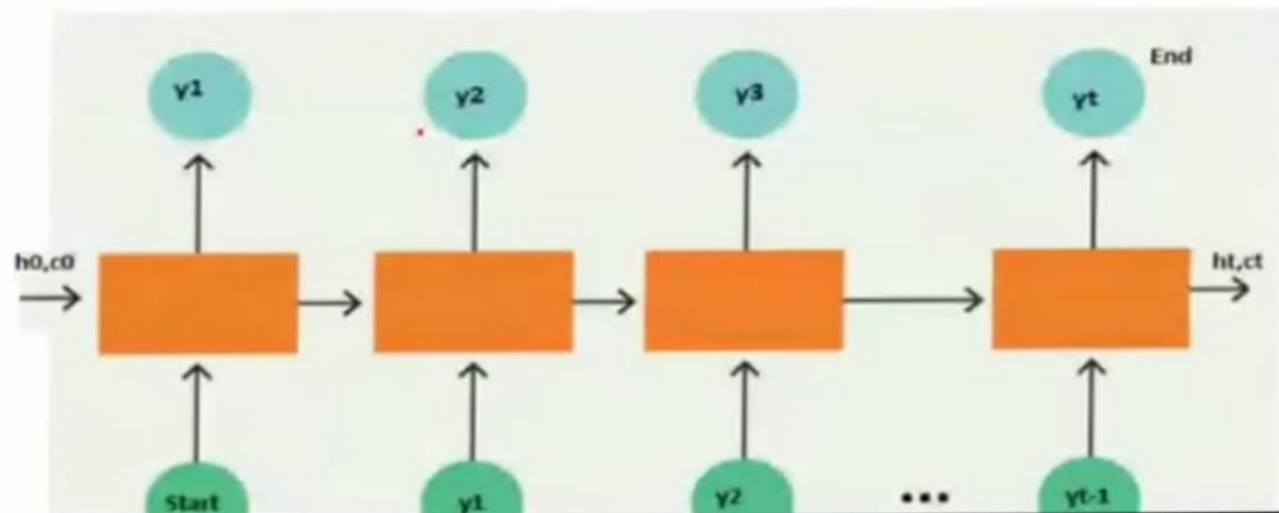
Each recurrent unit

Accepts :

1. a hidden state from the previous unit(h_{t-1})
2. previous layer output y_{t-1}

Produces:

1. an output, as well as
2. its own hidden state.



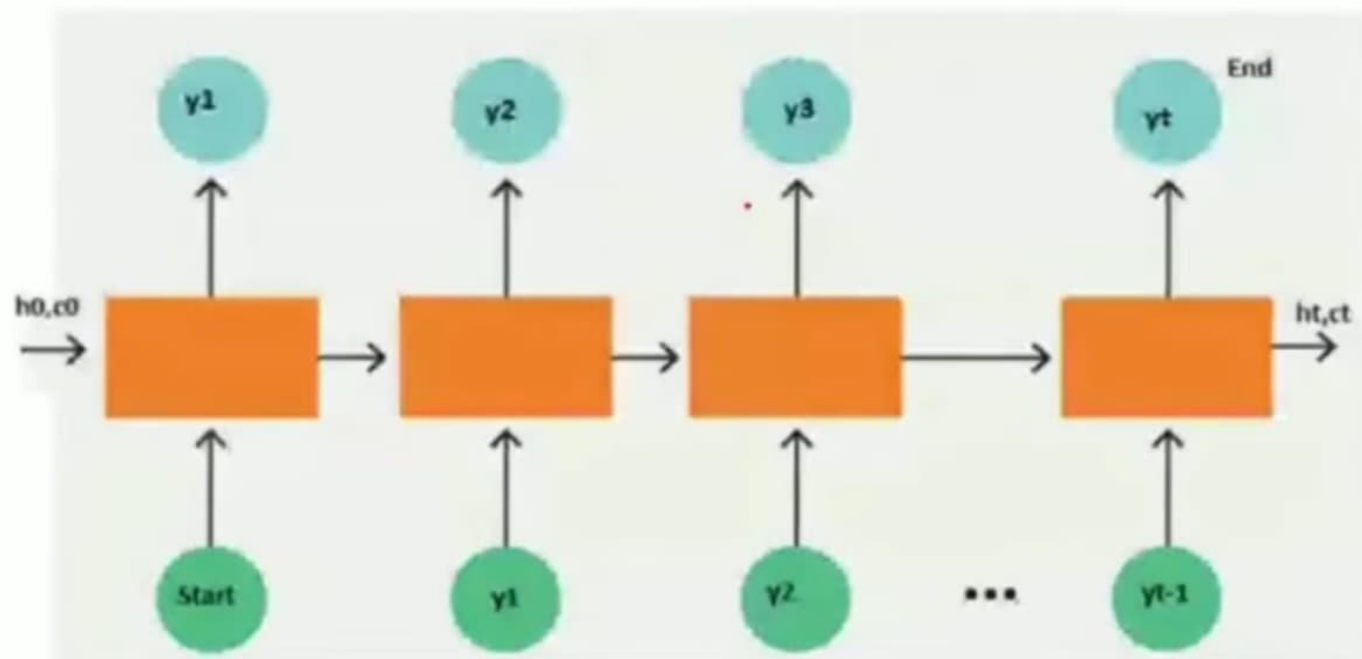
Any hidden state h_i is computed using the formula:

$$h_t = f(W^{(hh)} h_{t-1})$$

.

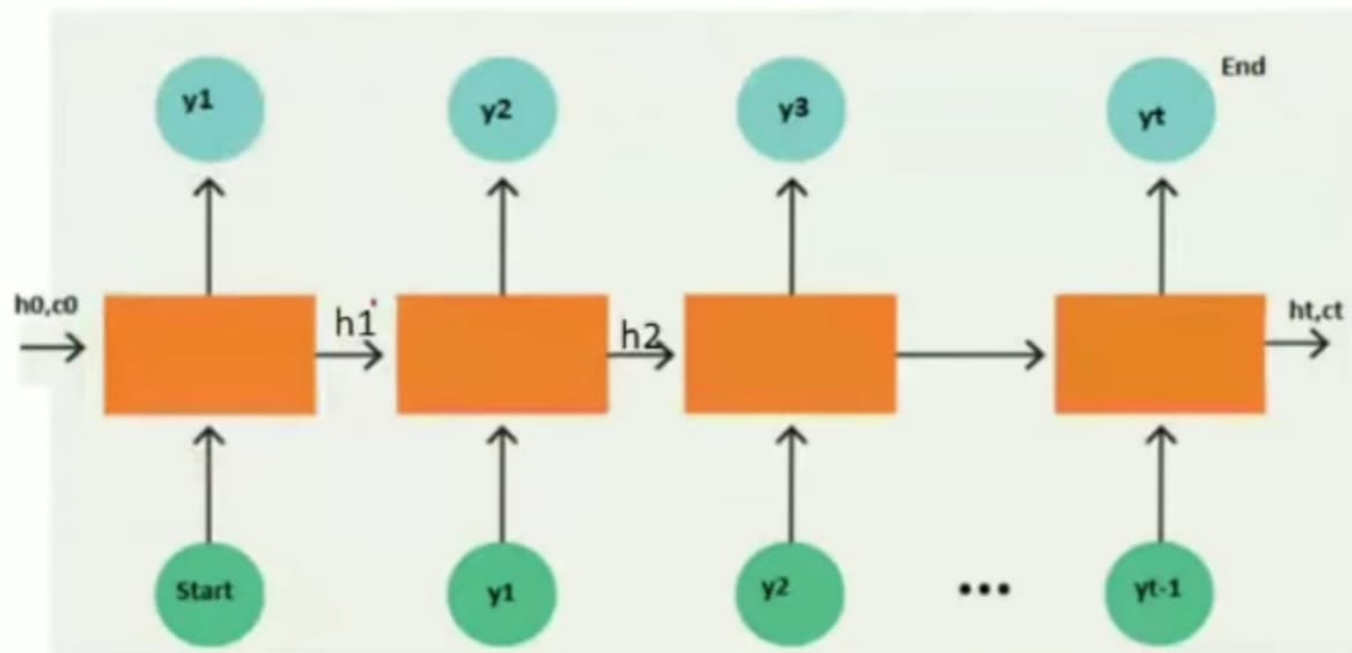
At the first layer,

- The output vector of encoder and the random symbol START,
- empty hidden state h_{t-1} will be **given as input**, t
- The **outputs** obtained will be y_1 and updated hidden state h_1 .



The second layer

- will have the updated hidden state h_1 and the previous output y_1 as **current inputs**, produces the hidden vector h_2 and output y_2 .



- The outputs occurred at each timestep of decoder is the **actual output**.
- The model will predict the output until the **END** symbol occurs.

Output Layer

- We use **Softmax activation** function at the output layer.
- It is used to produce the **probability distribution** from a vector of values with the target class of **high probability**.

The **output** y_t at time step t is computed using the formula:

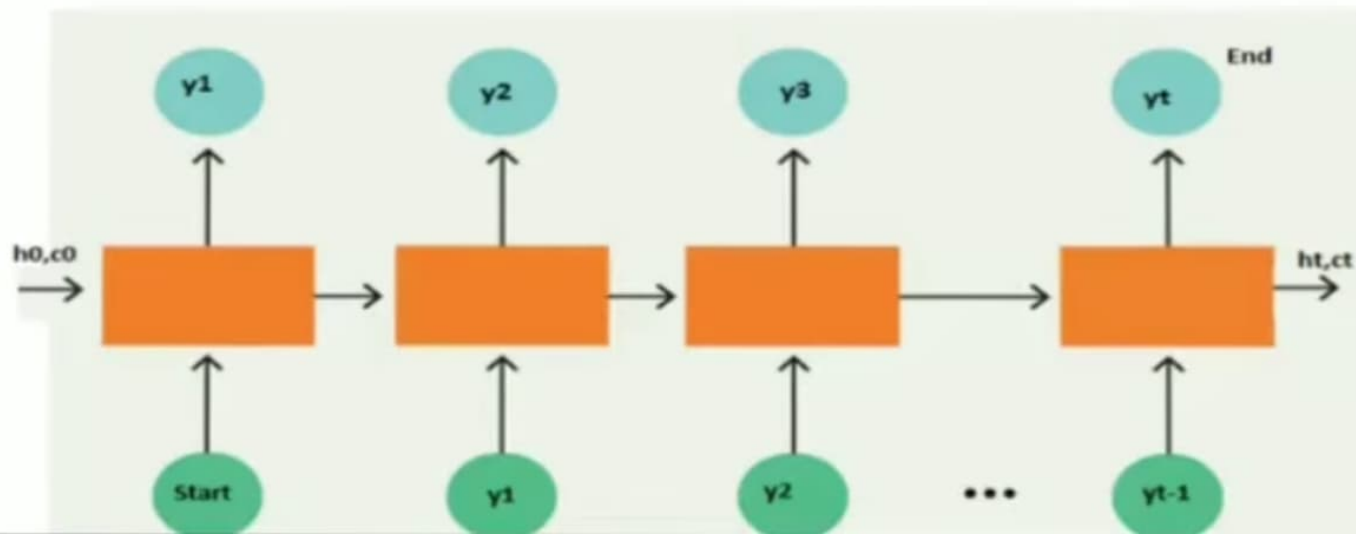
$$y_t = \textit{softmax}(W^S h_t)$$

- We **calculate the outputs** using the **hidden state** at the **current time step** together with the respective **weight** $W(S)$.

Example:

"**START**_John is hard working_**END**".

- The most important point is that the **initial states** (h_0, c_0) of the **decoder** are set to the **final states** of the **encoder**.
- This intuitively means that the **decoder** is **trained** to start generating the **output sequence** depending on the **information encoded by the encoder**.



- Finally, the **loss** is calculated on the **predicted outputs** from each time step and the **errors** are **backpropagated through time** in order to update the parameters of the network.
- Training the network over a **longer period** with a sufficiently **large** amount of data results in pretty **good predictions**.